
Reliability is Designed, Not Tested

Using a Temperature Monitoring
Device as an example



The System

Temperature Monitoring Device

Responsibilities:

- ▶ Read sensor
- ▶ Process data
- ▶ Display temperature
- ▶ Report measurement

Looks simple...

Until things go wrong.

What Can Go Wrong?

Real-world failures happen:

- ▶ Sensor timeout
- ▶ Invalid readings
- ▶ Communication failure
- ▶ Software hang
- ▶ Unexpected system state

Typical Reaction

Many teams focus on:



Feature Implementation



Functional testing



Assume reliability will follow

Unfortunately it doesn't.

Reliability Starts With architecture?

Good-architecture answer:

- ▶ How are failures detected?
- ▶ How are failure isolated?
- ▶ How is recovery performed?
- ▶ What happen after recovery fails?

Reliability Techniques?

We'll explore:

- ▶ HAL (Hardware Abstraction Layer)?
- ▶ Retry Mechanisms?
- ▶ Watchdog Pattern?
- ▶ State Machines?

Coming Next....

Part 2:

Hardware Abstraction Layer (HAL):

- ▶ How abstraction improves reliability, testability, and fault isolation.
- ▶ Full UML diagrams and C++ examples available on GitHub.